

VIVA STUDY GUIDE

NumPy · Pandas · Scikit-Learn

Complete Code Reference & Exam Preparation

1. NumPy — Numerical Python

NumPy is the foundational library for numerical computing in Python. It provides fast, efficient multi-dimensional arrays and mathematical functions.

1.1 Import & Setup

```
import numpy as np
```

💡 Always import NumPy as 'np' — this is the universal convention.

1.2 Creating Arrays

1-D Array

```
arr1 = np.array([1, 2, 3, 4, 5])
print(arr1)           # [1 2 3 4 5]
print(type(arr1))     # <class 'numpy.ndarray'>
```

2-D Array (Matrix)

```
arr2 = np.array([[1, 2, 3], [4, 5, 6]])
print(arr2.ndim)      # 2
print(arr2.shape)     # (2, 3) → 2 rows, 3 columns
```

3-D Array

```
arr3 = np.array([[[1,2,3],[4,5,6]], [[7,8,9],[10,11,12]]])
print(arr3.shape)     # (2, 2, 3)
```

💡 *ndim* = number of dimensions. *shape* = tuple of (rows, cols, depth...).

1.3 Array Indexing

Operation	Code Example	Result
1-D index	<code>arr[0]</code>	First element
2-D index	<code>arr[0, 1]</code>	Row 0, Column 1
3-D index	<code>arr[0, 1, 2]</code>	Block 0, Row 1, Col 2
Negative index	<code>arr[-1]</code>	Last element

Code Examples

```
arr = np.array([20, 2, 3, 4])
print(arr[0])           # 20
sum = arr[0] + arr[2]   # 20 + 3 = 23
```

```
arr2d = np.array([[1,2,3],[6,7,8]])
print(arr2d[0, 1])      # 2 (row 0, col 1)
```

1.4 Array Slicing

```
arr = np.array([1, 2, 3, 4, 5, 65, 7])
arr[1:6]    # index 1 to 5 → [2 3 4 5 65]
arr[5:]     # from index 5 to end → [65 7]
arr[:5]     # from start to index 4 → [1 2 3 4 5]
arr[:]      # entire array
```

💡 *Slicing syntax: arr[start:stop] — stop is EXCLUSIVE (not included).*

1.5 Data Types (.dtype)

```
arr1 = np.array([1, 2, 3, 4])
print(arr1.dtype)    # int64
```

```
arr2 = np.array(['apple', 'banana', 'cherry'])
print(arr2.dtype)    # <U6 (Unicode string)
```

1.6 Copy vs View — IMPORTANT!

This is a very common exam question!

Copy (independent — changes do NOT affect original)

```
arr = np.array([1, 2, 3, 4, 5])
x = arr.copy()
arr[0] = 42
print('main Array:', arr)    # [42 2 3 4 5]
print('Copy array:', x)     # [1 2 3 4 5] ← unchanged
```

View (linked — changes AFFECT both)

```
arr = np.array([1, 2, 3, 4, 5])
x = arr.view()
arr[0] = 42
print(arr)    # [42 2 3 4 5]
print(x)      # [42 2 3 4 5] ← ALSO changed!
```

💡 *copy() = independent clone. view() = just another reference to same data.*

1.7 Reshape

```
arr = np.array([1,2,3,4,5,6,7,8,9,10,11,12])
```

```
# 1-D → 2-D: 4 rows, 3 columns
newarr = arr.reshape(4, 3)
```

```
# 1-D → 3-D: 2 blocks, 3 rows, 2 columns
newarr2 = arr.reshape(2, 3, 2)
```

💡 *Total elements must match: 12 = 4×3 = 2×3×2*

1.8 Join & Split

Join (Concatenate)

```
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
arr = np.concatenate((arr1, arr2))
# Result: [1 2 3 4 5 6]
```

Split

```
arr = np.array([1, 2, 3, 4, 5, 6])
newarr = np.array_split(arr, 3)
# Result: [array([1,2]), array([3,4]), array([5,6])]
```

1.9 Searching

```
arr = np.array([1, 2, 3, 4, 5, 4, 4])
x = np.where(arr == 4)
# Returns: (array([3, 5, 6]),) ← indices where value == 4
```

```
# Find even numbers:
y = np.where(arr % 2 == 0)
```

```
# Find odd numbers:
z = np.where(arr % 2 == 1)
```

💡 *np.where() returns a TUPLE of arrays with the matching indices.*

1.10 Sorting

```
arr = np.array([3, 2, 0, 1])
print(np.sort(arr))      # [0 1 2 3]
```

```
arr1 = np.array(['banana', 'cherry', 'apple'])
print(np.sort(arr1))     # ['apple' 'banana' 'cherry']
```

```
arr2 = np.array([[3,2,4],[5,0,1]])
print(np.sort(arr2))     # sorts each row independently
```

💡 *np.sort() returns a NEW sorted array — does NOT modify the original.*

1.11 Shuffle

```
from numpy import random
arr = np.array([1, 2, 3, 4, 5])
random.shuffle(arr)      # Shuffles IN-PLACE (modifies original)
print(arr)               # e.g., [3 1 5 2 4]
```

1.12 Arithmetic Operations

Function	Code	Description
Add	<code>np.add(arr1, arr2)</code>	Element-wise addition
Subtract	<code>np.subtract(arr1, arr2)</code>	Element-wise subtraction
Multiply	<code>np.multiply(arr1, arr2)</code>	Element-wise multiplication
Divide	<code>np.divide(arr1, arr2)</code>	Element-wise division
Remainder	<code>np.mod(arr1, arr2)</code>	Element-wise modulo (remainder)
Absolute	<code>np.absolute(arr)</code>	Absolute value of each element

Code Example

```
arr1 = np.array([10, 11, 12, 16, 14, 15])
arr2 = np.array([2, 11, 6, 8, 2, 3])
np.add(arr1, arr2)      # [12 22 18 24 16 18]
np.subtract(arr1, arr2) # [8 0 6 8 12 12]
np.multiply(arr1, arr2) # [20 121 72 128 28 45]
np.divide(arr1, arr2)   # [5.0 1.0 2.0 2.0 7.0 5.0]
np.mod(arr1, arr2)      # [0 0 0 0 0 0]
```

1.13 Rounding Functions

Function	Code	Example Result	Description
<code>trunc()</code>	<code>np.trunc([-3.1666, 3.6667])</code>	<code>[-3. 3.]</code>	Removes decimal (toward zero)
<code>fix()</code>	<code>np.fix([-3.1666, 3.6667])</code>	<code>[-3. 3.]</code>	Same as trunc
<code>around()</code>	<code>np.around(3.1666, 2)</code>	<code>3.17</code>	Round to N decimals
<code>floor()</code>	<code>np.floor([-3.1666, 3.6667])</code>	<code>[-4. 3.]</code>	Round DOWN
<code>ceil()</code>	<code>np.ceil([-3.1666, 3.6667])</code>	<code>[-3. 4.]</code>	Round UP

1.14 NumPy Quick Reference Cheat Sheet

Concept	Code	Notes
Import	<code>import numpy as np</code>	
1-D array	<code>np.array([1,2,3])</code>	
2-D array	<code>np.array([[1,2],[3,4]])</code>	
Dimensions	<code>arr.ndim</code>	Number of axes
Shape	<code>arr.shape</code>	Tuple e.g. (rows, cols)
Data type	<code>arr.dtype</code>	int64, float64, <U6...
Index	<code>arr[0] / arr[0,1]</code>	1D / 2D access
Slice	<code>arr[1:5]</code>	Elements at index 1,2,3,4
Copy	<code>arr.copy()</code>	Independent clone

Concept	Code	Notes
View	<code>arr.view()</code>	Shared memory reference
Reshape	<code>arr.reshape(2,3)</code>	Must have same total elements
Join	<code>np.concatenate((a,b))</code>	Merges arrays
Split	<code>np.array_split(arr, 3)</code>	Splits into 3 equal parts
Search	<code>np.where(arr==4)</code>	Returns tuple of indices
Sort	<code>np.sort(arr)</code>	Returns new sorted array
Shuffle	<code>random.shuffle(arr)</code>	In-place, no return value

2. Pandas — Data Analysis Library

Pandas is built on top of NumPy and provides two key data structures: Series (1-D) and DataFrame (2-D table). It is the go-to library for data manipulation and analysis.

2.1 Import & Setup

```
import pandas as pd # 'pd' is the universal alias
```

2.2 Pandas Series

A Series is a 1-dimensional labeled array. Think of it as a single column from a spreadsheet.

From a List

```
a = [1, 7, 2]
myvar = pd.Series(a)
print(myvar.to_string())
# 0    1
# 1    7
# 2    2
print(myvar[0]) # 1 ← access by index
```

With Custom Labels

```
a = [1, 7, 2]
myvar = pd.Series(a, index=['x', 'y', 'z'])
print(myvar['x']) # 1 ← access by label
```

From a Dictionary

```
calories = {'day1': 420, 'day2': 380, 'day3': 390}
myvar = pd.Series(calories)
# Keys become the index, values become the data
```

 *Series = 1 column. DataFrame = full table with multiple columns.*

2.3 Pandas DataFrame

A DataFrame is a 2-dimensional table — like a spreadsheet or SQL table with rows and columns.

From a Dictionary

```
data = {
    'calories': [420, 380, 390],
    'duration': [50, 40, 45]
}
df = pd.DataFrame(data)
print(df)
#    calories  duration
# 0      420      50
```

```
# 1      380      40
# 2      390      45
```

From a CSV File

```
df = pd.read_csv('data.csv')
print(df.to_string()) # Print all rows
```

From a Nested Dictionary

```
data = {
    'Duration': {'0':60, '1':60, '2':60, '3':45},
    'Pulse':    {'0':110, '1':117, '2':103, '3':109},
    'Calories': {'0':409, '1':479, '2':340, '3':282}
}
df = pd.DataFrame(data)
```

💡 In nested dictionaries: outer keys = column names, inner keys = row indices.

2.4 Viewing Data

```
df.head(5) # First 5 rows (default is 5 if no argument)
df.tail()  # Last 5 rows
df.head(10) # First 10 rows
```

💡 head() and tail() are great for quickly checking your data after loading.

2.5 Handling Missing Data

```
df = pd.read_csv('data.csv')
new_df = df.dropna() # Remove all rows with any NaN
print(new_df.to_string())
```

Method	Code	Description
dropna()	df.dropna()	Remove rows with missing values
fillna()	df.fillna(0)	Replace NaN with a value (e.g., 0)
isnull()	df.isnull()	Boolean mask of missing values
to_string()	df.to_string()	Print all rows (no truncation)

2.6 Pandas Quick Reference Cheat Sheet

Concept	Code	Notes
Import	import pandas as pd	
Series from list	pd.Series([1,7,2])	1-D labeled array
Series with labels	pd.Series(a, index=['x','y','z'])	Custom index
Series from dict	pd.Series({'a':1,'b':2})	Keys = index
DataFrame from dict	pd.DataFrame({'col1':[1,2],'col2':[3,4]})	

Concept	Code	Notes
Read CSV	<code>pd.read_csv('file.csv')</code>	Load file into DataFrame
First N rows	<code>df.head(N)</code>	Default N=5
Last N rows	<code>df.tail(N)</code>	Default N=5
Print all rows	<code>df.to_string()</code>	No truncation
Remove NaN rows	<code>df.dropna()</code>	Returns new DataFrame
Access element	<code>myvar[0]</code> or <code>myvar['label']</code>	By position or label

3. Scikit-Learn — Machine Learning Library

Scikit-learn (sklearn) is the most popular Python ML library. It provides tools for classification, regression, clustering, preprocessing, and model evaluation — all with a consistent, easy-to-use API.

3.1 Installation & Import

```
# Install (run in terminal or notebook):  
!pip install -U scikit-learn
```

```
import sklearn  
print('Scikit-learn version:', sklearn.__version__)
```

3.2 Loading a Dataset — Iris Dataset

The Iris dataset is the classic beginner ML dataset. It contains measurements of 150 iris flowers across 3 species.

```
from sklearn.datasets import load_iris
```

```
iris = load_iris()
```

```
X = iris.data          # Features (input) — shape (150, 4)  
y = iris.target        # Labels (output) — 0, 1, or 2
```

```
feature_names = iris.feature_names  
target_names  = iris.target_names
```

```
print('Feature names:', feature_names)  
# ['sepal length (cm)', 'sepal width (cm)',  
#  'petal length (cm)', 'petal width (cm)']
```

```
print('Target names:', target_names)  
# ['setosa', 'versicolor', 'virginica']
```

```
print('Type of X:', type(X))    # <class 'numpy.ndarray'>  
print('First 5 rows:\n', X[:5])
```

Variable	Contains	Shape
X = iris.data	Sepal/Petal measurements (features)	(150, 4)
y = iris.target	Species label: 0, 1, or 2	(150,)
iris.feature_names	Names of the 4 input columns	List of 4 strings
iris.target_names	['setosa', 'versicolor', 'virginica']	List of 3 strings

3.3 Splitting the Dataset — train_test_split

Before training, we split data into training set (to learn from) and test set (to evaluate on).

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.4,          # 40% for testing, 60% for training
    random_state=1          # Seed for reproducibility
)
```

```
print('X_train Shape:', X_train.shape) # (90, 4)
print('X_test Shape:', X_test.shape)   # (60, 4)
print('Y_train Shape:', y_train.shape) # (90,)
print('Y_test Shape:', y_test.shape)   # (60,)
```

💡 test_size=0.4 means 40% testing / 60% training. random_state ensures you get the same split every time.

3.4 Handling Categorical Data — Encoding

Machine learning models need NUMBERS, not text. Encoding converts text categories into numbers.

Label Encoding — converts each category to a single integer

```
from sklearn.preprocessing import LabelEncoder

categorical_feature = ['cat', 'dog', 'dog', 'cat', 'bird']

encoder = LabelEncoder()
encoded_feature = encoder.fit_transform(categorical_feature)

print('Encoded feature:', encoded_feature)
# [1 2 2 1 0] ← bird=0, cat=1, dog=2 (alphabetical)
```

One-Hot Encoding — creates a binary column for each category

```
from sklearn.preprocessing import OneHotEncoder
import numpy as np

categorical_feature = ['cat', 'dog', 'dog', 'cat', 'bird']
categorical_feature = np.array(categorical_feature).reshape(-1, 1)

encoder = OneHotEncoder(sparse_output=False)
encoded_feature = encoder.fit_transform(categorical_feature)

print('OneHotEncoded feature:\n', encoded_feature)
# [[0. 1. 0.] ← cat
#  [0. 0. 1.] ← dog
#  [0. 0. 1.] ← dog]
```

```
# [0. 1. 0.] ← cat
# [1. 0. 0.] ← bird
```

Encoding Type	Input	Output	When to Use
Label Encoding	['cat','dog','bird']	[1, 2, 0]	Ordinal data (has natural order)
One-Hot Encoding	['cat','dog','bird']	[[0,1,0],[0,0,1],[1,0,0]]	Nominal data (no natural order)

💡 *One-Hot avoids the problem where Label Encoding implies order (bird < cat < dog is meaningless!).*

3.5 Training the Model — Logistic Regression

```
from sklearn.linear_model import LogisticRegression
```

```
log_reg = LogisticRegression(max_iter=200)
log_reg.fit(X_train, y_train) # Train on training data
```

💡 *max_iter=200 increases the iteration limit to help the model converge (find a solution).*

3.6 Evaluating the Model

```
from sklearn import metrics
```

```
# Predict on test data:
y_pred = log_reg.predict(X_test)
```

```
# Make predictions on new samples:
sample = [[3, 5, 4, 2], [2, 3, 5, 4]]
preds = log_reg.predict(sample)
pred_species = [iris.target_names[p] for p in preds]
print('Predictions:', pred_species)
```

```
# Model accuracy:
print('Accuracy:', metrics.accuracy_score(y_test, y_pred))
```

💡 *accuracy_score compares predicted labels (y_pred) vs actual labels (y_test) and returns a value between 0 and 1.*

3.7 The ML Workflow — Summary

Step	Action	Code
1. Load Data	Load dataset into X and y	iris = load_iris()
2. Split	Train/test split	train_test_split(X, y, test_size=0.4)
3. Encode (if needed)	Convert text to numbers	LabelEncoder() / OneHotEncoder()
4. Train	Fit model on training data	model.fit(X_train, y_train)

Step	Action	Code
5. Predict	Make predictions	<code>model.predict(X_test)</code>
6. Evaluate	Check accuracy	<code>metrics.accuracy_score(y_test, y_pred)</code>

3.8 Scikit-Learn Quick Reference Cheat Sheet

Concept	Code	Notes
Install	<code>!pip install -U scikit-learn</code>	
Load Iris	<code>from sklearn.datasets import load_iris</code>	Classic practice dataset
Split data	<code>train_test_split(X, y, test_size=0.4, random_state=1)</code>	40% test / 60% train
Label Encode	<code>LabelEncoder().fit_transform(data)</code>	Single integer per class
One-Hot Encode	<code>OneHotEncoder(sparse_output=False).fit_transform(data)</code>	Binary columns per class
Logistic Regression	<code>LogisticRegression(max_iter=200)</code>	Classification model
Train model	<code>model.fit(X_train, y_train)</code>	Learning step
Predict	<code>model.predict(X_test)</code>	Inference step
Accuracy	<code>metrics.accuracy_score(y_test, y_pred)</code>	0.0 to 1.0 score

4. Master Comparison: NumPy vs Pandas vs Scikit-Learn

Feature	NumPy	Pandas	Scikit-Learn
Purpose	Numerical computation	Data analysis & manipulation	Machine Learning
Core data type	ndarray	Series / DataFrame	Model objects
Import alias	import numpy as np	import pandas as pd	from sklearn import ...
Data shape	arr.shape, arr.ndim	df.shape	X.shape
Key strength	Fast math on arrays	Table manipulation, CSV	Training & evaluating models
Built on top of	(Core library)	NumPy	NumPy + SciPy

5. Common Viva Questions & Answers

Q1: What is the difference between `copy()` and `view()` in NumPy?

`copy()` creates an independent array — changes to the original don't affect the copy. `view()` creates a reference to the same data — changes to one affect the other.

Q2: What is `ndim` vs `shape`?

`ndim` gives the NUMBER of dimensions (e.g., 2 for a 2-D array). `shape` gives the SIZE of each dimension as a tuple (e.g., (3, 4) means 3 rows and 4 columns).

Q3: What is the difference between Label Encoding and One-Hot Encoding?

Label Encoding assigns a single integer to each category (bird=0, cat=1, dog=2). This implies order. One-Hot Encoding creates a separate binary column for each category — better for nominal (unordered) data.

Q4: What does `train_test_split` do?

It randomly splits data into training and testing sets. The model learns from `X_train/y_train` and is evaluated on `X_test/y_test`. `test_size=0.4` means 40% for testing. `random_state` ensures the same split every run.

Q5: What is a Pandas Series vs DataFrame?

A Series is 1-dimensional (like a single column). A DataFrame is 2-dimensional (like a full table with rows and columns). A DataFrame is essentially a collection of Series.

Q6: Why use `np.where()` and what does it return?

`np.where(condition)` finds the indices where a condition is True. It returns a tuple of arrays — one array per dimension. Example: `np.where(arr==4)` returns `(array([3, 5, 6]),)` if value 4 is at indices 3, 5, and 6.

Q7: What is the Iris dataset?

The Iris dataset contains 150 flower samples with 4 features each (sepal length, sepal width, petal length, petal width). The target is one of 3 species: setosa, versicolor, or virginica. It is used to practice classification algorithms.

Q8: What does max_iter mean in Logistic Regression?

max_iter is the maximum number of iterations the solver runs to find the best model parameters. The default is 100, but sometimes the model needs more iterations to converge. Setting max_iter=200 gives it more time.

Good luck on your Viva! You've got this! 🎓